Machine Learning for Molecular Engineering
3/7/10/20.01 (U) 3/7/10/20.C51 (G)
Spring 2025

Problem Set #6

Date: May 5, 2025
Due: May 12, 2025 @ 5 pm

Instructions

- This PSET is only for undergraduate students to complete, and, as such, is 75 points in total.
- To get started, open your Google Colab or Jupyter notebook starting from the problem set template file pset6.ipynb (direct Colab link). You will need to use the data files here. **Make your own copy of this template to save changes, by selecting "Save a copy in Drive".**
- **Important:** This problem set requires a GPU. Before you start in Google Colab, find `Notebook Settings` under the `Edit` menu. In the `Hardware accelerator` drop-down, select a T4 GPU as your hardware accelerator. Changing the runtime resets the notebook, so make this change before starting!
- Collaboration is encouraged and AI tools are permitted, but submitting work that is not your own is plagiarism. Any collaboration or assistance from an LLM (including utilities present within Google Colab) should be described at the end of your submission.
- Upon completion, submit your IPython Notebook `pset6.ipynb` to the non-bio assignment on Gradescope. Ensure your submission includes *all* necessary code to be run without error, with all plots and outputs. Comments are encouraged; put answers to conceptual questions in Markdown/Text cells.

Background

MACE is an equivariant neural network interatomic potential (NNIP) architecture, which researchers have used across a number of chemical domains to predict the energies and forces of various atomic systems [1]. MACE is an equivariant architecture, meaning that when an input is rotated or reflected, the resulting directional properties, such as forces, are constrained to undergo the same transformation. Researchers have trained MACE on large swaths of different chemical spaces in an effort to offer "foundational" models tailored for problems in that space. The MACE model pretrained on the Materials Project data, consisting of 150k organic crystals, is referred to as MACE-MP-0; the MACE model that is trained on 1 million conformers from organic and biomolecular systems (e.g. solvated amino acids, amino-acid ligand pairs, etc) is known as MACE-OFF [2]. There are many versions and sizes of pretrained MACE models, which can be found in the MACE-foundation and MACE-OFF libraries.

Here, we will use MACE to refer to the general strategy of using a NNIP or the model library/architecture itself, and MACE-MP-0 or MACE-OFF to refer to the pretrained models we will employ

(in this pset, we will use only the `medium` sizes of both models). Throughout the PSET, you will find referring to the MACE documentation very helpful.

Our goal for this PSET will be to first explore molecular conformers and see if we can leverage a pretrained MACE model, which have demonstrated impressive performance on simple electronic and quantum properties, to learn protein-ligand binding energy, ΔG_{bind}.

$$\Delta G = E(\text{protein and ligand bound}) - E(\text{protein and ligand unbound})$$

Though this is still a change in energy, rather than energy itself, we hypothesize that we can utilize the MACE architecture and pretraining information to learn effectively from a small subset of data. Unfortunately, protein-ligand systems are unlikely to fit in memory on a single GPU, so to simplify matters (and increase the difficulty of the learning task for MACE), we will select only one protein system–PFKFB3–and use the few ligands reported for which we have both conformer information and experimental binding affinity, which has been converted into a ΔG for our use case [3]. We will refrain from supplying the protein or solvent atomic coordinates to speed up prediction as using all these relevant coordinates is too memory-intensive to load on a single GPU for training or inference.

After exploring some of the conformer data in part 1, you will train three different models and compare performance in part 2: a MACE model from scratch, a finetuned pretrained MACE-OFF model, and a simple GNN/MLP head that takes MACE-learned descriptors as input. Part 3 will explore as an extension task running molecular dynamics with MACE as the force field instead of a more semi-empirical method.

Part 1: Exploring conformer distribution and utilizing MACE to look at energies (25 points)

Part 1.1 Visualize molecule conformers & datasets (7.5 points)

After downloading the dataset, use `pybel` and `py3Dmol` to load and visualize your conformers within the context of the protein pocket that they dock in.

Task 1: Visualize the conformers using `py3Dmol`, alongside the protein structure for reference (set a lower opacity for the PDB structure only). To get the "block" representation, you can use the `.write(format)` function of a `pybel` molecule object.

Task 2: Describe some of the visual differences you see amongst the conformers in this dataset. What features look preserved? What atoms of the pocket appear to be in interaction with the molecules? Do you see any possible correlations between observed molecular motifs and experimental ΔG?

Part 1.2 Compute energies of conformers with MACE (5 points)

Task: Use the MACE-MP-0 and MACE-OFF pretrained models to come up with energies for each of your conformers, using the `mace.calculators` method. Plot a scatterplot between each of the molecules and report any differences, for individual molecules and about the general distribution of the data.

Part 1.3 Plotting atom-level descriptors/features as derived from MACE (12.5 points)

Task: As you may recall from lecture and PSET 3, GNNs build atom-specific features over successive layers; while these are usually aggregated in some manner to predict a molecule-level property,

we can nonetheless extract and utilize the atom-level features as possibly informative embeddings.

MACE also offers such atom-level embeddings, known as descriptors, which you can read about more https://mace-docs.readthedocs.io/en/latest/guide/descriptors.html.

Task 1: Following the documentation above, let's compare the utility of the MACE-OFF vs. MACE-MP-0 descriptors. Collect the physical descriptors as generated by both MACE-OFF and MACE-MP-0 separately (using `invariants_only=True`). Average embeddings over all atoms per molecule (so you have one embedding per molecule), and try clustering them with UMAP. Color the samples based on their experimental ΔG value.

Task 2: Answer the following questions:

1. What do you observe in terms of the clustering captured by the UMAP embeddings relative to the ΔG values? Do certain clusters capture a subset of ΔG well? Why do you think this might be the case?

2. Does one model outperform the other based on the quality of clustering or separation? What information could be useful in aiding the model to perform better on out of distribution data given atomic features?

3. Based on the quality of separation, what do you think the performance of building a regressor from these embeddings will be?

Part 2: Part 2: Comparing different strategies for transfer learning and finetuning with MACE (40 points)

In these experiments, you'll work through three distinct training configurations—both to familiarize yourself with varied learning strategies and to give you the opportunity to try testing different model configurations using the MACE documentation as a guide.

Part 2.1 Make train/validation/test splits for training (5 points)

Task: Create a 70%-10%-20% training-validation-test split on the ligand dataset. The first part of preprocessing the SDF entries into .xyz formatting suitable for the `ase` library has been provided.

Part 2.2 Train MACE from scratch (10 points)

First, let's use the basic MACE architecture to try predicting binding energy from the conformer data. The MACE library offers a handy CLI interface for training models, which we will utilize.

Task 1: Set up a `config_from_scratch.yml`, which has the specifications for keyword arguments you'd like to pass to MACE. An example of one such training config, which we will need to tweak, is provided in Figure 1.

To this sample config, let's make some changes. You're encouraged to look through the MACE documentation and the argument parsing code to make the right choice of settings for your model.

- Task parameters: we do not have forces or stresses to train on, so any force weights need to be set to 0/our loss should not have any force or stress terms. Consider setting the `forces_weight`, `stresses_weight`, and the `loss` accordingly. MACE also incorporates *stochastic weight averaging* with `swa`, which typically trains a different model but with a separate set of energy/force weights. You can either change the swa weights or turn it off. We will also need to provide isolated atom energies, which we can ask MACE to pre-compute empirically

```
%%writefile config/config-03.yml

model: "MACE"
num_channels: 32
max_L: 0
r_max: 4.0
name: "mace02_com1"
model_dir: "MACE_models"
log_dir: "MACE_models"
checkpoints_dir: "MACE_models"
results_dir: "MACE_models"
train_file: "data/solvent_xtb_train_20.xyz"
valid_file: "data/solvent_xtb_train_50.xyz"
test_file: "data/solvent_xtb_test.xyz"
energy_key: "energy_xtb"
forces_key: "forces_xtb"
device: cuda
batch_size: 5
max_num_epochs: 300
swa: True
seed: 123
```

Figure 1: Example of MACE config file.

from our dataset with `E0s: "average"`. Because we are computing per-molecule energies, our final reporting of error can also be adjusted to not report a RMSE per atom by picking a different option for `error_table`.

- MACE configuration: There are two choices of MACE that might be relevant to our use case `MACE` or `ScaleShiftMACE`, the latter of which will shift energies by mean energies and forces computed from the dataset. Since we do not have forces, though, stick with `MACE`.
- Training parameters: make sure to update the `train/valid/test_file` parameters to be correct references; set the `max_num_epochs` to be 100 and `batch_size` to be 10.

Task 2: After training, use the `eval_mace` function and `aseMolec.pltProps`, `aseMolec.extAtoms` library to help you plot a scatterplot, RMSE, and R^2 for each of the train, validation, and test predictions from the baseline model. Label all axes and titles appropriately.

Part 2.3 Finetune MACE-OFF on the same dataset (5 points)

Now, we'll try finetuning a copy of the MACE-OFF model. You can reuse much of the same config settings from your first model, though you'll need to `name` your model distinctly from the first to avoid overwriting your earlier training.
Task 1: Finetune the MACE-OFF model (you can find the weights path from where it was downloaded for Part 1.2), by creating a `config_finetune.yml` and using the `train_mace` function. You will need to additionally set the `foundation_model` and `multiheads_finetuning` parameters. Since finetuning should already have sufficient learning of weights, train for only 50 epochs.
Task 2: After training, use the `eval_mace` function and `aseMolec.pltProps`, `aseMolec.extAtoms` library to help you plot a scatterplot, RMSE, and R^2 for each of the train, validation, and test predictions from the finetuned model. Label all axes and titles appropriately.

Part 2.4 Train mini GNN/MLP on MACE descriptors (20 points)

With the descriptors we collected in Part 1.3, we can also if we can train a simple regressor head on these descriptors to see if they capture sufficient information. For some other property prediction tasks [4], MACE descriptors have been found quite informative!

Part 2.4.1 Preliminary questions (5 points)

1. What are the conceptual differences between finetuning and using these descriptors as our input?
2. What might be the benefits of designing a predictive model separate from the MACE architecture?

Part 2.4.2 Build a mini GNN/MLP architecture to predict ΔG (10 points)

Let's build a simple MLP that can operate on the atom embeddings. Using your non-averaged, **per-atom** MACE-OFF embeddings from 1.3, build a train/validation/test split, DataLoaders (of batch size 4) for your data, and a simple GNN/MLP architecture as follows:

1. (at least) one GCNConv layer between atoms that preserves the size of the embedding.
2. an MLP, composed of two linear layers with requisite nonlinearities, that operates on each embedding individually
3. a pooling step to aggregate per-atom embeddings into a single value.

Feel free to refer back to the code you wrote in PSET 3 to help you with this!

Part 2.4.3 Plot scatterplot of predictions (5 points)

After training, plot a scatterplot, RMSE, and R^2 for each of the train, validation, and test predictions from the descriptors model. Label all axes and titles appropriately.

Part 2.5 Overall evaluation and observations (5 points)

We just tried three different strategies for training! Report some of the differences you see in performance, any outliers or handicaps you observe in quality, and what you think overall the best strategy could be. Finally, as a conceptual question, if we had multiple protein-ligand systems that we wanted to try using the MACE architecture to train on, what would be your preferred strategy of learning binding energy?

Part 3: Run MD simulations with the MACE-OFF potential (5 points)

As a final task, let's also explore how one can use a learned potential (e.g. MACE-OFF or MACE-MP-0) to compute molecular dynamics simulations. Typically, these are done with physics-based force fields which can be at times slow to run and therefore prohibitive to running large-scale simulations. Neural network potentials like MACE-OFF offer the opportunity to accelerate these simulations, though the sacrifice in accuracy can vary across different systems and configurations.

Let's try running a simulation! Load the `1aou.pdb` into the starting configuration, and then following the instructions here, set up a Langevin simulation to run for 500 timesteps with an interval of 25. Visualize the trajectory using a tool of your choice (e.g. PyMol locally, which you can install here), and comment on any changes you see visually occurring over the course of the trajectory.

References

[1] Batatia, I. *et al.* A foundation model for atomistic materials chemistry. *arXiv [physics.chem-ph]* (2023).

[2] Kovács, D. P. *et al.* MACE-OFF: Transferable short range machine learning force fields for organic molecules. *arXiv [physics.chem-ph]* (2023).

[3] Ross, G. A. *et al.* The maximal and current accuracy of rigorous protein-ligand binding free energy calculations. *Commun. Chem.* **6**, 222 (2023).

[4] Shiota, T., Ishihara, K. & Mizukami, W. Universal neural network potentials as descriptors: towards scalable chemical property prediction using quantum and classical computers. *Digit. Discov.* **3**, 1714–1728 (2024).